

Data-Driven Optimization

9주차

Traveling Salesman Problem – 2

OR-Tools

예제 1: 여행사 영업 담당자 문제

<https://developers.google.com/optimization/routing/tsp>



예제 1: 여행사 영업 담당자 문제 – code (1) - 데이터 생성

```
from ortools.constraint_solver import routing_enums_pb2
from ortools.constraint_solver import pywrapcp
```

def create_data_model():

```
    """Stores the data for the problem."""
```

```
    data = {}
```

```
    data["distance_matrix"] = [
        [0, 2451, 713, 1018, 1631, 1374, 2408, 213, 2571, 875, 1420, 2145, 1972],
        [2451, 0, 1745, 1524, 831, 1240, 959, 2596, 403, 1589, 1374, 357, 579],
        [713, 1745, 0, 355, 920, 803, 1737, 851, 1858, 262, 940, 1453, 1260],
        [1018, 1524, 355, 0, 700, 862, 1395, 1123, 1584, 466, 1056, 1280, 987],
        [1631, 831, 920, 700, 0, 663, 1021, 1769, 949, 796, 879, 586, 371],
        [1374, 1240, 803, 862, 663, 0, 1681, 1551, 1765, 547, 225, 887, 999],
        [2408, 959, 1737, 1395, 1021, 1681, 0, 2493, 678, 1724, 1891, 1114, 701],
        [213, 2596, 851, 1123, 1769, 1551, 2493, 0, 2699, 1038, 1605, 2300, 2099],
        [2571, 403, 1858, 1584, 949, 1765, 678, 2699, 0, 1744, 1645, 653, 600],
        [875, 1589, 262, 466, 796, 547, 1724, 1038, 1744, 0, 679, 1272, 1162],
        [1420, 1374, 940, 1056, 879, 225, 1891, 1605, 1645, 679, 0, 1017, 1200],
        [2145, 357, 1453, 1280, 586, 887, 1114, 2300, 653, 1272, 1017, 0, 504],
        [1972, 579, 1260, 987, 371, 999, 701, 2099, 600, 1162, 1200, 504, 0],
    ]
```

```
    data["num_vehicles"] = 1
```

```
    data["depot"] = 0
```

```
    return data
```

- data["num_vehicles"]는 차량 대수
- 앞의 문제에서는 차량 1만 고려했으나 2대 이상의 차량에 대한 모델링도 가능함.

예제 1: 여행사 영업 담당자 문제 - code (2)

```
data = create_data_model()
```

```
# 색인 관리자, 일종의 정보관리자
```

```
manager = pywrapcp.RoutingIndexManager(  
    len(data["distance_matrix"]), data["num_vehicles"], data["depot"]  
)
```

```
# Routing 객체 생성
```

```
routing = pywrapcp.RoutingModel(manager)
```

```
# 두 점 사이의 거리를 반환한다.
```

```
def distance_callback(from_index, to_index):  
    # Convert from routing variable Index to distance matrix NodeIndex.  
    from_node = manager.IndexToNode(from_index)  
    to_node = manager.IndexToNode(to_index)  
    return data[["distance_matrix"]][from_node][to_node]
```

```
# 거리 계산 함수를 callback 함수로 등록하고 활용함.
```

```
transit_callback_index = routing.RegisterTransitCallback(distance_callback) ← 함수 등록
```

```
routing.SetArcCostEvaluatorOfAllVehicles(transit_callback_index) ← 솔버에게 거리 계산 방법을 지정해 줌.
```

- 솔버 내부 인덱스를 사용자 위치번호로 변환하는 등 인덱스를 관리함.
- RoutingIndexManager의 입력자료로는 거리 행렬, 차량 숫자, 출발지를 포함함.

예제 1: 여행사 영업 담당자 문제 (3)

시작해를 찾기 위한 휴리스틱 메소드를 등록

```
search_parameters = pywrapcp.DefaultRoutingSearchParameters()
search_parameters.first_solution_strategy = (
    routing_enums_pb2.FirstSolutionStrategy.PATH_CHEAPEST_ARC
)
```

시작점으로부터 가까운 노드를 추가하는 방식
다른 전략은 여기에...
https://developers.google.com/optimization/routing/routing_options?hl=ko#first_solution_strategy

```
def print_solution(manager, routing, solution):
    """콘솔에 해 출력"""
    print(f"Objective: {solution.ObjectiveValue()} miles")
    index = routing.Start(0)
    plan_output = "Route for vehicle 0:\n"
    route_distance = 0
    while not routing.IsEnd(index):
        plan_output += f" {manager.IndexToNode(index)} ->"
        previous_index = index
        index = solution.Value(routing.NextVar(index))
        route_distance += routing.GetArcCostForVehicle(previous_index, index, 0)
    plan_output += f" {manager.IndexToNode(index)}\n"
    print(plan_output)
    plan_output += f"Route distance: {route_distance}miles\n"
```

예제 1: 여행사 영업 담당자 문제 (4)

문제 풀이

```
solution = routing.SolveWithParameters(search_parameters)
if solution:
    print_solution(manager, routing, solution)
```

```
Objective: 7293 miles
Route for vehicle 0:
0 -> 7 -> 2 -> 3 -> 4 -> 12 -> 6 -> 8 -> 1 -> 11 -> 10 -> 5 -> 9 -> 0
```

해 경로를 리스트에 저장하기 (Optional)

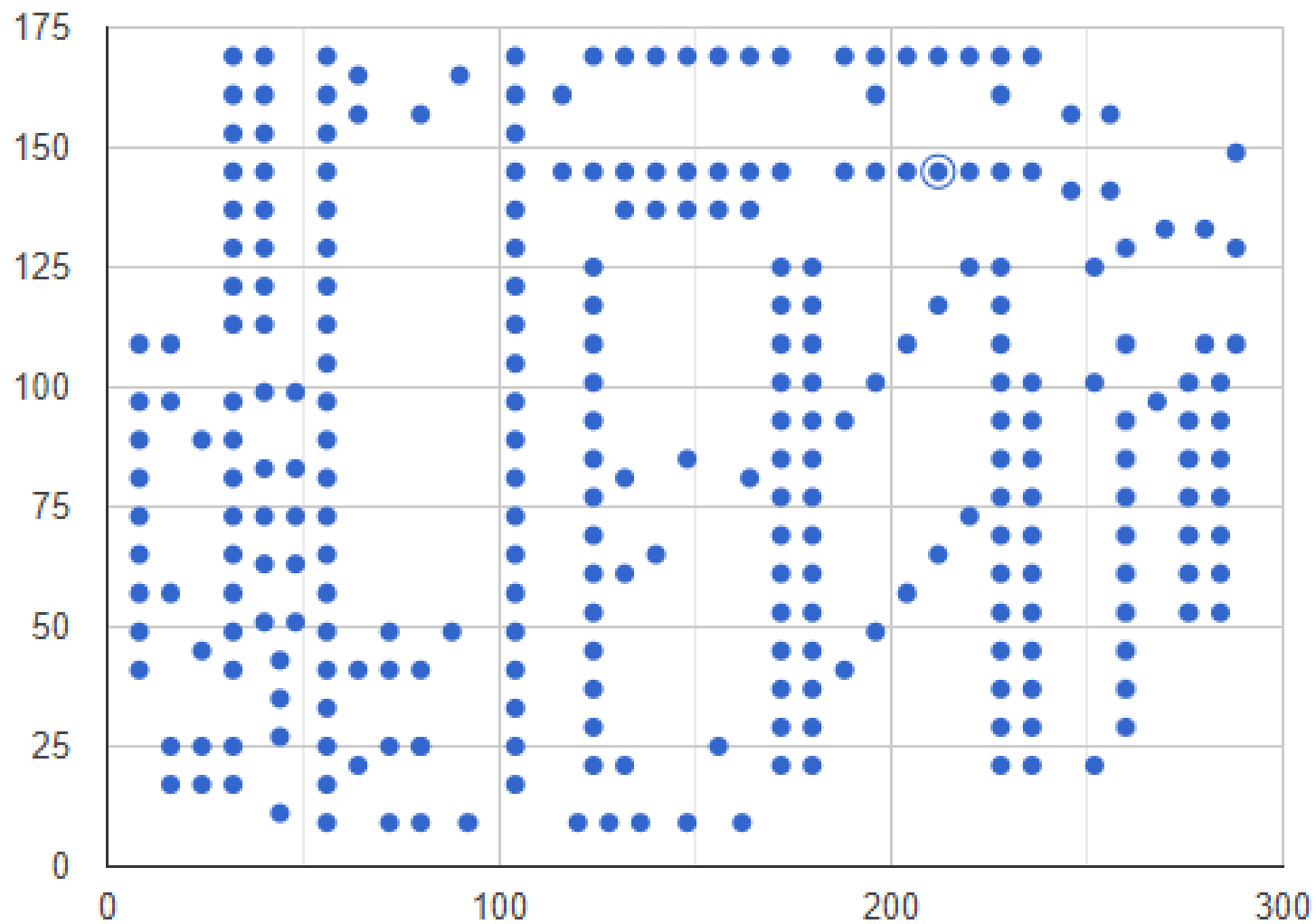
```
def get_routes(solution, routing, manager):
    """Get vehicle routes from a solution and store them in an array."""
    # Get vehicle routes and store them in a two dimensional array whose
    # i,j entry is the jth location visited by vehicle i along its route.
    routes = []
    for route_nbr in range(routing.vehicles()):
        index = routing.Start(route_nbr)
        route = [manager.IndexToNode(index)]
        while not routing.IsEnd(index):
            index = solution.Value(routing.NextVar(index))
            route.append(manager.IndexToNode(index))
        routes.append(route)
    return routes
```

```
routes = get_routes(solution, routing, manager)
```

Display the routes.

```
for i, route in enumerate(routes):
    print('Route', i, route)
```

예제 2: 기판 드릴링



- a280.tsp
- u1432.tsp

예제 2: 기관 드릴링 – coding (지난 주 코드 활용)

```
from ortools.constraint_solver import routing_enums_pb2
from ortools.constraint_solver import pywrapcp
import math
import pandas as pd

#%%
def makeDIST(nP):
    DIST = list()
    nCity = len(nP)

    for i in range(nCity):
        DIST.append([])
        for j in range(nCity):
            if j != i:
                # Euclidean Distance
                temp = math.hypot(nP['xc'][i] - nP['xc'][j], nP['yc'][i] - nP['yc'][j])
                DIST[i].append(int(temp))
            else:
                DIST[i].append(0)
    return DIST
```

```
def create_data_model(DIST):
    """Stores the data for the problem."""
    data = {}
    data["distance_matrix"] = DIST
    data["num_vehicles"] = 1
    data["depot"] = 0
    return data

def print_solution(manager, routing, solution):
    """Prints solution on console."""
    print(f"Objective: {solution.ObjectiveValue()} miles")
    index = routing.Start(0)
    plan_output = "Route for vehicle 0:\n"
    route_distance = 0
    while not routing.IsEnd(index):
        plan_output += f" {manager.IndexToNode(index)} ->"
        previous_index = index
        index = solution.Value(routing.NextVar(index))
        route_distance += routing.GetArcCostForVehicle(previous_index, index, 0)
    plan_output += f" {manager.IndexToNode(index)}\n"
    print(plan_output)
    plan_output += f"Route distance: {route_distance}miles\n"
```

예제 2: 기판 드릴링 - coding

```
# Main Flow
f = open("TSP_data\\a280.tsp", 'r') # 기판
nPos = pd.DataFrame(columns=[ 'xc', 'yc'])

flag = 0
while True:
    line = f.readline().strip()
    if line == "EOF": break

    if line == "NODE_COORD_SECTION":
        flag = 1
        continue

    if flag:
        ss = line.split()
        nPos.loc[len(nPos)] = [float(ss[1]), float(ss[2])]

f.close()

# Draw a picture
import matplotlib.pyplot as plt
import numpy as np

plt.scatter(nPos['xc'], nPos['yc'])
plt.show()
```

파일 편집 보기

```
NAME : a280
COMMENT : drilling problem (Ludwig)
TYPE : TSP
DIMENSION: 280
EDGE_WEIGHT_TYPE : EUC_2D
NODE_COORD_SECTION
1 288 149
2 288 129
3 270 133
4 256 141
5 256 157
6 246 157
7 236 169
8 228 169
9 228 161
10 220 169
11 212 169
12 204 169
13 196 169
14 188 169
15 196 161
16 188 145
17 172 145
18 164 145
19 156 145
20 148 145
21 140 145
22 148 169
23 164 169
24 172 169
```

예제 2: 기판 드릴링 - coding

RoutingIndexManager는 사용자 정의 노드 번호(node index)와 내부적으로 사용하는 라우팅 인덱스(routing index) 간의 매핑을 관리하는 클래스

```
DIST = makeDIST(nPos)
```

거리 행렬 만들기

```
data = create_data_model(DIST)
```

```
manager = pywrapcp.RoutingIndexManager(  
    len(data["distance_matrix"]), data["num_vehicles"], data["depot"]  
)
```

색인 관리자

```
# Create Routing Model.
```

```
routing = pywrapcp.RoutingModel(manager)
```

routing model 생성

```
def distance_callback(from_index, to_index):  
    from_node = manager.IndexToNode(from_index)  
    to_node = manager.IndexToNode(to_index)  
    return data["distance_matrix"][from_node][to_node]
```

거리 계산 함수를
callback 함수로
등록하고 활용함.

```
transit_callback_index = routing.RegisterTransitCallback(distance_callback)
```

```
routing.SetArcCostEvaluatorOfAllVehicles(transit_callback_index)
```

```
search_parameters = pywrapcp.DefaultRoutingSearchParameters()  
search_parameters.first_solution_strategy = (  
    routing_enums_pb2.FirstSolutionStrategy.PATH_CHEAPEST_ARC  
)
```

초기해 생성.

```
solution = routing.SolveWithParameters(search_parameters)
```

```
# Print solution on console.
```

```
if solution:
```

```
    print_solution(manager, routing, solution)
```

예제 2: 기판 드릴링 - coding

```
277 240 141
278 252 125
279 260 129
280 280 133
EOF
Objective: 2784 miles
Route for vehicle 0:
0 -> 1 -> 279 -> 2 -> 278 -> 277 -> 248 -> 247 -> 243 -> 242 -> 241 -> 240 -> 239 -> 238 -> 245 -> 244 -> 246 -> 249 -> 250 ->
> 229 -> 228 -> 231 -> 230 -> 237 -> 236 -> 235 -> 234 -> 233 -> 232 -> 227 -> 226 -> 225 -> 224 -> 223 -> 222 -> 218 -> 221 ->
> 220 -> 219 -> 202 -> 203 -> 204 -> 205 -> 207 -> 206 -> 211 -> 212 -> 215 -> 216 -> 217 -> 214 -> 213 -> 210 -> 209 -> 208 ->
> 251 -> 254 -> 255 -> 257 -> 256 -> 253 -> 252 -> 139 -> 140 -> 141 -> 142 -> 143 -> 199 -> 201 -> 200 -> 195 -> 194 -> 193 ->
> 191 -> 190 -> 189 -> 188 -> 187 -> 163 -> 164 -> 165 -> 166 -> 167 -> 168 -> 169 -> 171 -> 170 -> 172 -> 105 -> 106 -> 104 ->
> 103 -> 107 -> 109 -> 111 -> 110 -> 113 -> 114 -> 116 -> 117 -> 61 -> 62 -> 63 -> 65 -> 64 -> 84 -> 85 -> 115 -> 112 -> 86 ->
83 -> 82 -> 87 -> 81 -> 80 -> 88 -> 108 -> 89 -> 90 -> 91 -> 102 -> 101 -> 100 -> 99 -> 98 -> 97 -> 96 -> 95 -> 94 -> 93 -> 9
2 -> 79 -> 78 -> 77 -> 76 -> 74 -> 75 -> 73 -> 72 -> 71 -> 70 -> 69 -> 66 -> 68 -> 67 -> 57 -> 56 -> 55 -> 54 -> 53 -> 52 -> 5
1 -> 50 -> 49 -> 48 -> 47 -> 46 -> 45 -> 44 -> 43 -> 58 -> 60 -> 59 -> 42 -> 41 -> 40 -> 39 -> 38 -> 37 -> 36 -> 35 -> 34 -> 3
3 -> 32 -> 31 -> 30 -> 29 -> 124 -> 123 -> 122 -> 121 -> 120 -> 119 -> 118 -> 156 -> 157 -> 158 -> 173 -> 162 -> 161 -> 160 ->
174 -> 159 -> 150 -> 151 -> 155 -> 152 -> 154 -> 153 -> 128 -> 129 -> 130 -> 131 -> 18 -> 19 -> 20 -> 127 -> 126 -> 125 -> 28
-> 27 -> 26 -> 25 -> 21 -> 24 -> 22 -> 23 -> 13 -> 12 -> 14 -> 11 -> 10 -> 9 -> 7 -> 8 -> 6 -> 5 -> 275 -> 274 -> 273 -> 272
-> 271 -> 270 -> 15 -> 16 -> 17 -> 132 -> 149 -> 177 -> 176 -> 175 -> 178 -> 179 -> 180 -> 181 -> 182 -> 183 -> 184 -> 186 ->
185 -> 192 -> 196 -> 197 -> 198 -> 144 -> 145 -> 146 -> 147 -> 148 -> 138 -> 137 -> 136 -> 135 -> 134 -> 133 -> 269 -> 268 ->
267 -> 266 -> 265 -> 264 -> 263 -> 262 -> 261 -> 260 -> 258 -> 259 -> 276 -> 3 -> 4 -> 0
```

사례: 폐기물 수거 시스템

자연상점

iTainer-W

IoT 컨테이너 수거시스템
생활폐기물용



iDry-B

음식폐기물
건조 감량기



iSolarbin

태양광 자동압축시스템



iTainer-R

IoT 컨테이너 수거시스템
재활용폐기물용



iTainer Robot

딥러닝 카메라 및 자동 선별 다관절 로봇
자동압축 및 배출 컨테이너



iMbin

자동압축 수거함
계량 및 과금 수거함



i Ecopro C : Ecology Professional in Collection
Nature Store : 자연을 사랑하는 상점을 의미합니다.

사례: 폐기물 수거 시스템 (2)

(의사결정 변수)

- 몇 대의 차량이 필요한가
- 각 차량의 운행 경로

(제약 조건)

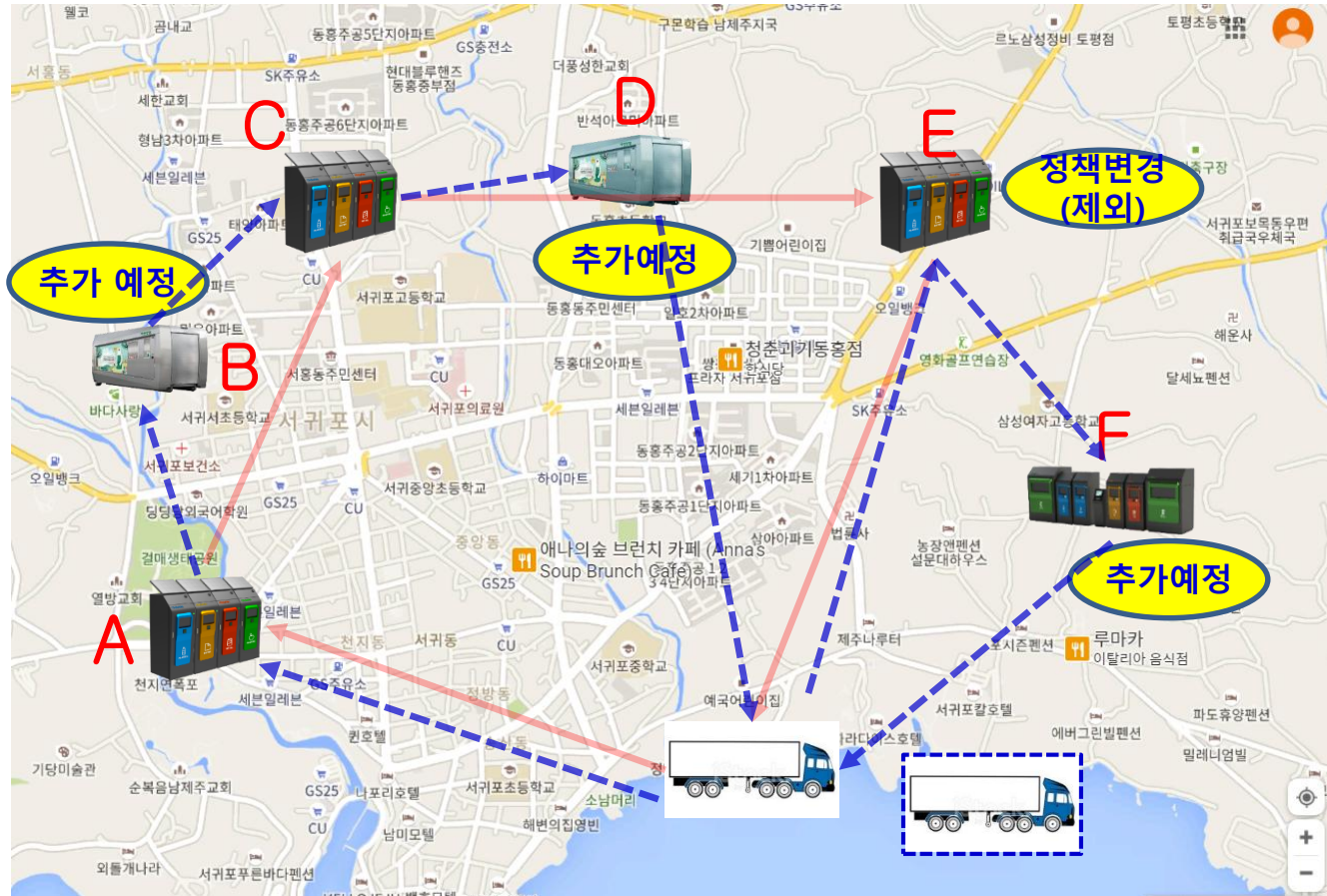
- 각 지점 하루 1회 이상 방문
- 최단거리 운행 차량과 최장 거리 운행 차량의 운행 거리 차 최소화

(목적함수)

- 총 거리 + (100) 최장 거리 운행 차량의 운행 거리

(추가 고려사항)

- 수거 시점 예측 후 방문 지점에 대한 수거 계획 문제
- 신설, 변경으로 문제 변경 가능



예제 3: 차량 라우팅(VRP, Vehicle Routing Problem) 문제

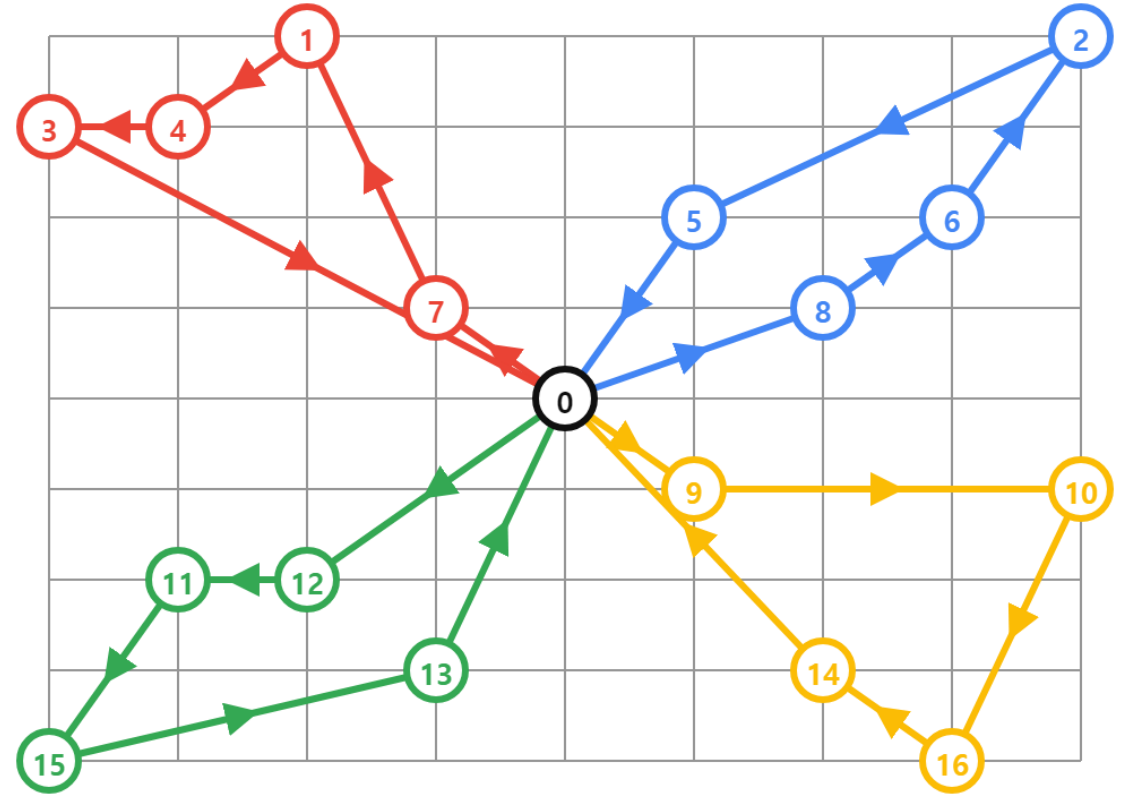
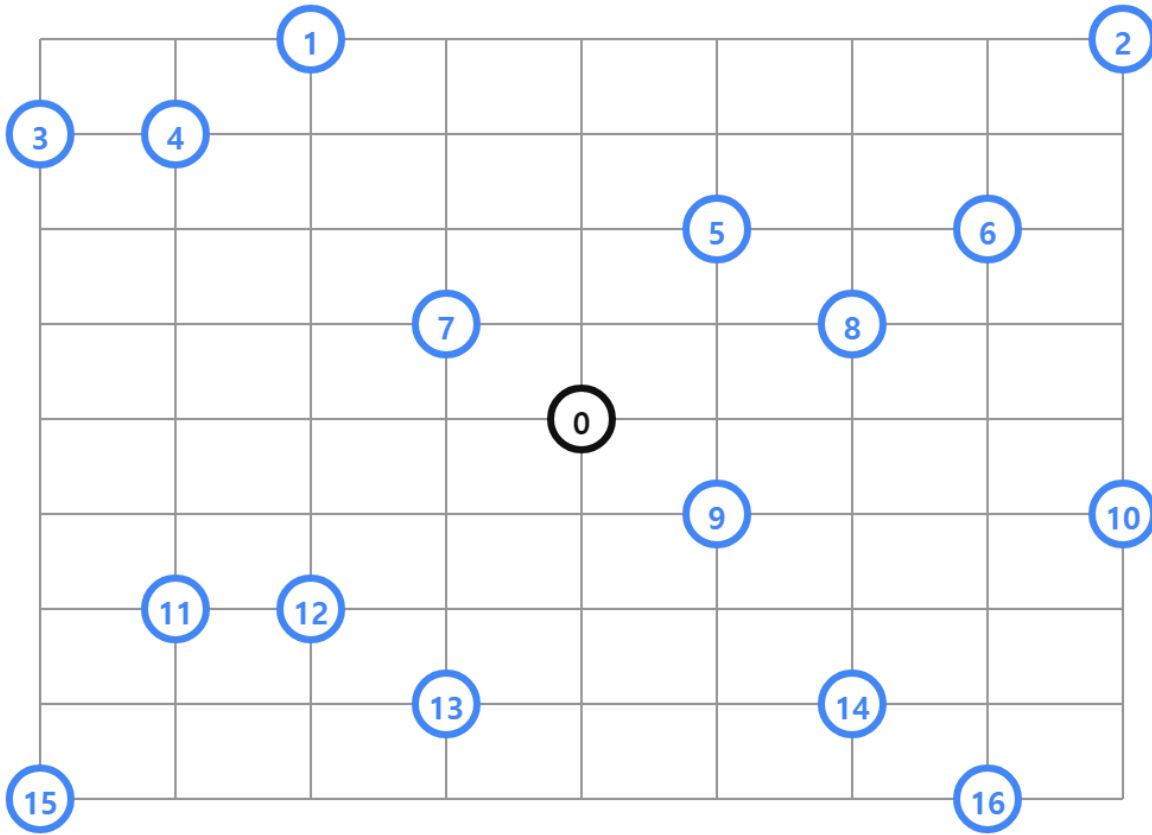
차량 라우팅 문제 (VRP)의 목표는 여러 위치를 여러 대의 차량으로 방문할 때 각 차량에 대한 최적의 경로를 찾는 문제

최적의 경로를 찾는 좋은 방법은 모든 차량 중에서 가장 긴 단일 경로의 길이를 최소화하는 것임. 최대한 빨리 모든 전송을 완료하는 것을 목표로 할 수 있음. VRP 예에서는 이러한 방식으로 정의된 최적의 경로를 찾음.

예제 3: 경로 최적화 문제 유형

1. Vehicle Routing Problem (VRP)
2. Capacity Constraint Vehicle Routing Problem (CVRP)
3. Vehicle Routing Problem with Time Windows (VRPTW)

1. Vehicle Routing Problem (VRP) - or9_1_vrp.py



- 차량 4 대

1. Vehicle Routing Problem (VRP) - 예제 코드

<https://developers.google.com/optimization/routing/vrp> 에서 복사

```
def create_data_model():
    """Stores the data for the problem."""
    data = {}
    data["distance_matrix"] = [
        # fmt: off
        [0, 548, 776, 696, 582, 274, 502, 194, 308, 194, 536, 502, 388, 354, 468, 776, 662],
        [548, 0, 684, 308, 194, 502, 730, 354, 696, 742, 1084, 594, 480, 674, 1016, 868, 1210],
        [776, 684, 0, 992, 878, 502, 274, 810, 468, 742, 400, 1278, 1164, 1130, 788, 1552, 754],
        [696, 308, 992, 0, 114, 650, 878, 502, 844, 890, 1232, 514, 628, 822, 1164, 560, 1358],
        [582, 194, 878, 114, 0, 536, 764, 388, 730, 776, 1118, 400, 514, 708, 1050, 674, 1244],
        [274, 502, 502, 650, 536, 0, 228, 308, 194, 240, 582, 776, 662, 628, 514, 1050, 708],
        [502, 730, 274, 878, 764, 228, 0, 536, 194, 468, 354, 1004, 890, 856, 514, 1278, 480],
        [194, 354, 810, 502, 388, 308, 536, 0, 342, 388, 730, 468, 354, 320, 662, 742, 856],
        [308, 696, 468, 844, 730, 194, 194, 342, 0, 274, 388, 810, 696, 662, 320, 1084, 514],
        [194, 742, 742, 890, 776, 240, 468, 388, 274, 0, 342, 536, 422, 388, 274, 810, 468],
        [536, 1084, 400, 1232, 1118, 582, 354, 730, 388, 342, 0, 878, 764, 730, 388, 1152, 354],
        [502, 594, 1278, 514, 400, 776, 1004, 468, 810, 536, 878, 0, 114, 308, 650, 274, 844],
        [388, 480, 1164, 628, 514, 662, 890, 354, 696, 422, 764, 114, 0, 194, 536, 388, 730],
        [354, 674, 1130, 822, 708, 628, 856, 320, 662, 388, 730, 308, 194, 0, 342, 422, 536],
        [468, 1016, 788, 1164, 1050, 514, 514, 662, 320, 274, 388, 650, 536, 342, 0, 764, 194],
        [776, 868, 1552, 560, 674, 1050, 1278, 742, 1084, 810, 1152, 274, 388, 422, 764, 0, 798],
        [662, 1210, 754, 1358, 1244, 708, 480, 856, 514, 468, 354, 844, 730, 536, 194, 798, 0],
        # fmt: on
    ]
    data["num_vehicles"] = 4
    data["depot"] = 0
    return data
```

1. Vehicle Routing Problem (VRP) - 예제 코드 (2)

- 앞에서와 같이 위치 사이의 거리를 반환하는 거리 콜백함수를 생성하여 솔버에 등록함.

```
def distance_callback(from_index, to_index):  
    """Returns the distance between the two nodes."""  
    # Convert from routing variable Index to distance matrix NodeIndex.  
    from_node = manager.IndexToNode(from_index)  
    to_node = manager.IndexToNode(to_index)  
    return data["distance_matrix"][from_node][to_node]  
  
transit_callback_index = routing.RegisterTransitCallback(distance_callback)  
routing.SetArcCostEvaluatorOfAllVehicles(transit_callback_index)
```

- 경로 문제에서 시간이나 중량 문제를 다룰 때 다음 dimension 객체를 생성, 설정하여 활용함.

```
dimension_name = "Distance"  
routing.AddDimension(  
    transit_callback_index,  
    0, # no slack  
    3000, # vehicle maximum travel distance  
    True, # start cumul to zero  
    dimension_name,  
)  
distance_dimension = routing.GetDimensionOrDie(dimension_name)  
distance_dimension.SetGlobalSpanCostCoefficient(100)
```

- Global Span = max 누적값(종료 지점들) - min 누적값(시작 지점들)으로 정의(가장 멀리 간 차량과 가장 적게 간 차량 간의 차이)
- 원래의 목적함수(예: 총 이동 거리) + 100 × (가장 멀리 간 차량의 거리 - 가장 적게 간 차량의 거리)

1. Vehicle Routing Problem (VRP) - 예제 코드 (2)

```
rs/okcs/Library/CloudStorage/Dropbox/04_강 의 /2025-1/00_데 이 터 기 반 최 적 화 /01_python_예 제
파일 /or10_2_2.py
Objective: 177500
Route for vehicle 0:
 0 -> 9 -> 10 -> 2 -> 6 -> 5 -> 0
Distance of the route: 1712m

Route for vehicle 1:
 0 -> 16 -> 14 -> 8 -> 0
Distance of the route: 1484m

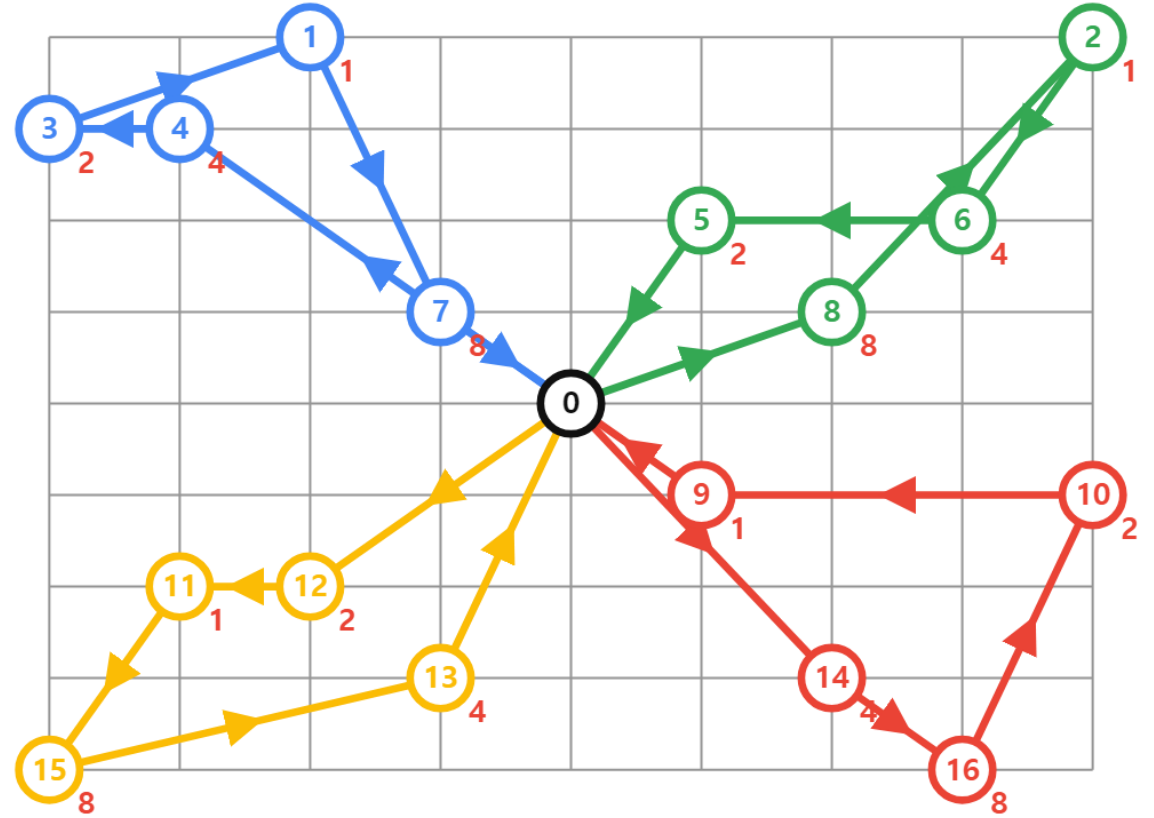
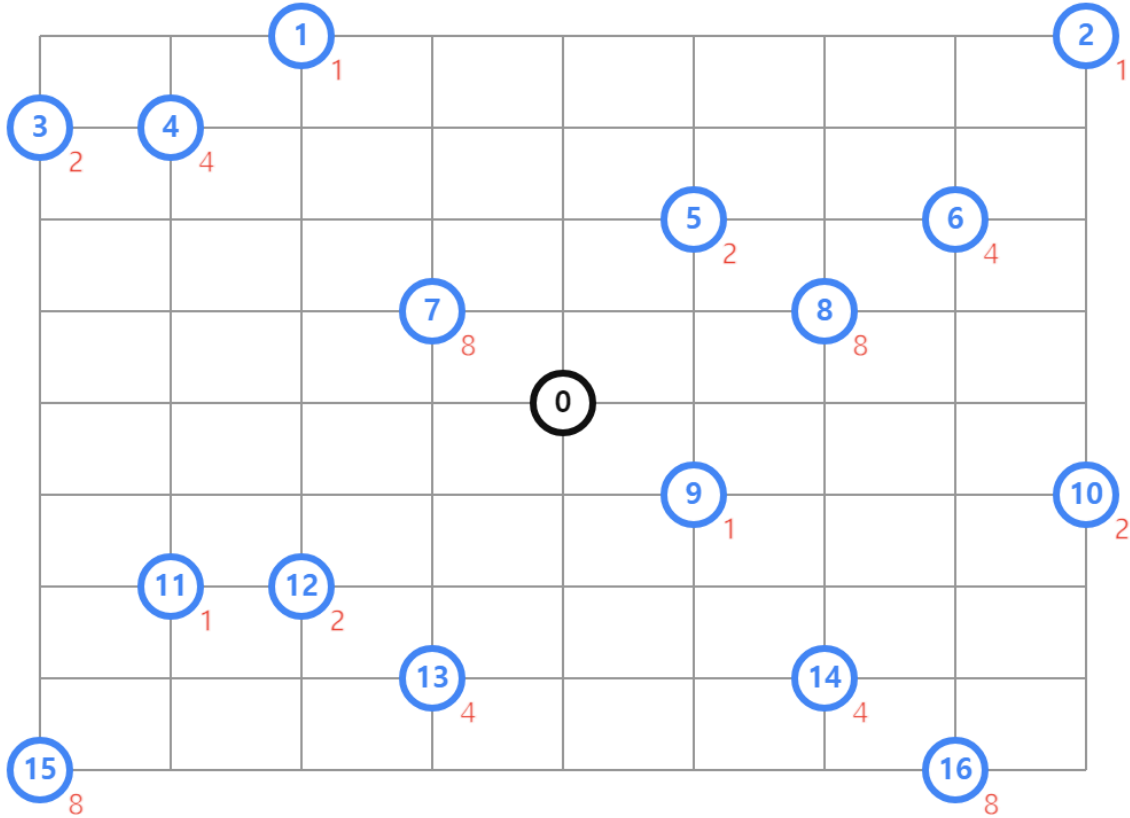
Route for vehicle 2:
 0 -> 7 -> 1 -> 4 -> 3 -> 0
Distance of the route: 1552m

Route for vehicle 3:
 0 -> 13 -> 15 -> 11 -> 12 -> 0
Distance of the route: 1552m

Maximum of the route distances: 1712m
❖ (base) okcs@ogchangsuui-MacBookPro 01_python_예 제 파 일 %
```

2. Capacity Constraint Vehicle Routing Problem (CVRP) - or9_2_cvrp.py

<https://developers.google.com/optimization/routing/cvrp?hl=ko>



- 차량 4 대
- 각 차량 15까지 운반 가능

2. Capacity Constraint Vehicle Routing Problem (CVRP)

```
data["demands"] = [0, 1, 1, 2, 4, 2, 4, 8, 8, 1, 2, 1, 2, 4, 4, 8, 8]
data["vehicle_capacities"] = [15, 15, 15, 15]
```

수요 콜백 및 용량 제약조건 추가

```
def demand_callback(from_index):
    """Returns the demand of the node."""
    # Convert from routing variable Index to demands NodeIndex.
    from_node = manager.IndexToNode(from_index)
    return data["demands"][from_node]
```

```
demand_callback_index = routing.RegisterUnaryTransitCallback(demand_callback)
routing.AddDimensionWithVehicleCapacity(
    demand_callback_index,
    0, # null capacity slack
    data["vehicle_capacities"], # vehicle maximum capacities
    True, # start cumul to zero
    "Capacity",
)
```

```
강 의 /2025-1/00_데이터 기반 최적화 /01_python_예제 파일 /or10_4.py
Objective: 6208
Route for vehicle 0:
 0 Load(0) -> 7 Load(8) -> 3 Load(10) -> 4 Load(14) -> 1 Load(15) -> 0 Load(15)
Distance of the route: 1552m
Load of the route: 15

Route for vehicle 1:
 0 Load(0) -> 14 Load(4) -> 16 Load(12) -> 10 Load(14) -> 9 Load(15) -> 0 Load(15)
Distance of the route: 1552m
Load of the route: 15

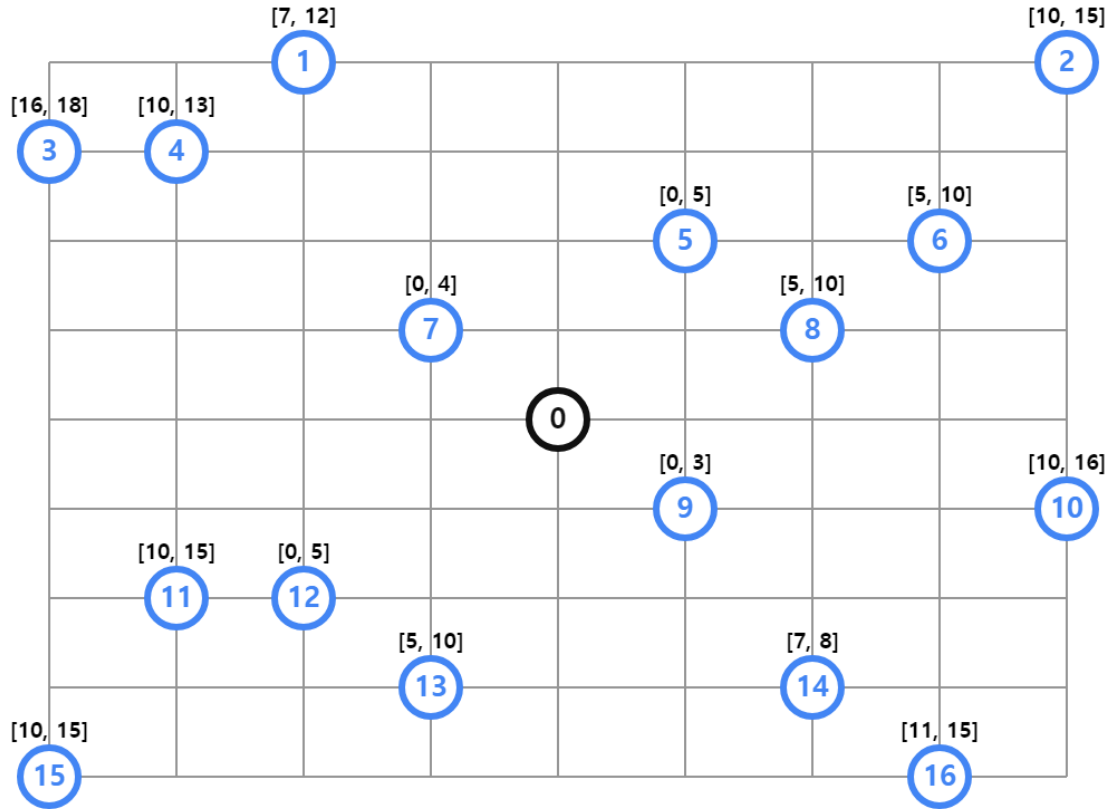
Route for vehicle 2:
 0 Load(0) -> 12 Load(2) -> 11 Load(3) -> 15 Load(11) -> 13 Load(15) -> 0 Load(15)
Distance of the route: 1552m
Load of the route: 15

Route for vehicle 3:
 0 Load(0) -> 8 Load(8) -> 2 Load(9) -> 6 Load(13) -> 5 Load(15) -> 0 Load(15)
Distance of the route: 1552m
Load of the route: 15

Total distance of all routes: 6208m
Total load of all routes: 60
```

3. Vehicle Routing Problem with Time Windows (VRPTW) - or9_3_vrptw.py

<https://developers.google.com/optimization/routing/vrptw?hl=ko>



Route for vehicle 0:

0 Time(0,0) -> 9 Time(2,3) -> 14 Time(7,8) -> 16 Time(11,11) -> 0 Time(18,18)

Time of the route: 18min

Route for vehicle 1:

0 Time(0,0) -> 7 Time(2,4) -> 1 Time(7,11) -> 4 Time(10,13) -> 3 Time(16,16) -> 0 Time(24,24)

Time of the route: 24min

Route for vehicle 2:

0 Time(0,0) -> 12 Time(4,4) -> 13 Time(6,6) -> 15 Time(11,11) -> 11 Time(14,14) -> 0 Time(20,20)

Time of the route: 20min

Route for vehicle 3:

0 Time(0,0) -> 5 Time(3,3) -> 8 Time(5,5) -> 6 Time(7,7) -> 2 Time(10,10) -> 10 Time(14,14) ->

0 Time(20,20)

Time of the route: 20min

Total time of all routes: 82min

- 차량 4 대
- 각 차량 15까지 운반 가능

3. Vehicle Routing Problem (VRP) - 예제 코드 (3)

```
def create_data_model():
    """Stores the data for the problem."""
    data = {}
    data["time_matrix"] = [
        [0, 6, 9, 8, 7, 3, 6, 2, 3, 2, 6, 6, 4, 4, 5, 9, 7],
        [6, 0, 8, 3, 2, 6, 8, 4, 8, 8, 13, 7, 5, 8, 12, 10, 14],
        [9, 8, 0, 11, 10, 6, 3, 9, 5, 8, 4, 15, 14, 13, 9, 18, 9],
        [8, 3, 11, 0, 1, 7, 10, 6, 10, 10, 14, 6, 7, 9, 14, 6, 16],
        [7, 2, 10, 1, 0, 6, 9, 4, 8, 9, 13, 4, 6, 8, 12, 8, 14],
        [3, 6, 6, 7, 6, 0, 2, 3, 2, 2, 7, 9, 7, 7, 6, 12, 8],
        [6, 8, 3, 10, 9, 2, 0, 6, 2, 5, 4, 12, 10, 10, 6, 15, 5],
        [2, 4, 9, 6, 4, 3, 6, 0, 4, 4, 8, 5, 4, 3, 7, 8, 10],
        [3, 8, 5, 10, 8, 2, 2, 4, 0, 3, 4, 9, 8, 7, 3, 13, 6],
        [2, 8, 8, 10, 9, 2, 5, 4, 3, 0, 4, 6, 5, 4, 3, 9, 5],
        [6, 13, 4, 14, 13, 7, 4, 8, 4, 4, 0, 10, 9, 8, 4, 13, 4],
        [6, 7, 15, 6, 4, 9, 12, 5, 9, 6, 10, 0, 1, 3, 7, 3, 10],
        [4, 5, 14, 7, 6, 7, 10, 4, 8, 5, 9, 1, 0, 2, 6, 4, 8],
        [4, 8, 13, 9, 8, 7, 10, 3, 7, 4, 8, 3, 2, 0, 4, 5, 6],
        [5, 12, 9, 14, 12, 6, 6, 7, 3, 3, 4, 7, 6, 4, 0, 9, 2],
        [9, 10, 18, 6, 8, 12, 15, 8, 13, 9, 13, 3, 4, 5, 9, 0, 9],
        [7, 14, 9, 16, 14, 8, 5, 10, 6, 5, 4, 10, 8, 6, 2, 9, 0],
    ]
```

```
data["time_windows"] = [
    (0, 5), # depot
    (7, 12), # 1
    (10, 15), # 2
    (16, 18), # 3
    (10, 13), # 4
    (0, 5), # 5
    (5, 10), # 6
    (0, 4), # 7
    (5, 10), # 8
    (0, 3), # 9
    (10, 16), # 10
    (10, 15), # 11
    (0, 5), # 12
    (5, 10), # 13
    (7, 8), # 14
    (10, 15), # 15
    (11, 15), # 16
]
data["num_vehicles"] = 4
data["depot"] = 0
return data
```

3. Vehicle Routing Problem (VRP) - 예제 코드 (3)

```
time = "Time"
routing.AddDimension(
    transit_callback_index,
    30, # allow waiting time
    30, # maximum time per vehicle
    False, # Don't force start cumul to zero.
    time,
)
time_dimension = routing.GetDimensionOrDie(time)
# Add time window constraints for each location except depot.
for location_idx, time_window in enumerate(data["time_windows"]):
    if location_idx == data["depot"]:
        continue
    index = manager.NodeToIndex(location_idx)
    time_dimension.CumulVar(index).SetRange(time_window[0], time_window[1])
# Add time window constraints for each vehicle start node.
depot_idx = data["depot"]
for vehicle_id in range(data["num_vehicles"]):
    index = routing.Start(vehicle_id)
    time_dimension.CumulVar(index).SetRange(
        data["time_windows"][depot_idx][0], data["time_windows"][depot_idx][1]
    )
for i in range(data["num_vehicles"]):
    routing.AddVariableMinimizedByFinalizer(
        time_dimension.CumulVar(routing.Start(i))
    )
    routing.AddVariableMinimizedByFinalizer(time_dimension.CumulVar(routing.End(i)))
```

```
Distance of the route: 1552m
Maximum of the route distances: 1712m
• (base) okcs@ogchangsuui-MacBookPro 01_python_예제파일 % /opt/anaconda3/bin/python /Users/okcs/Library/CloudStorage/Dropb
강 의 /2025-1/00_데이터기반최적화/01_python_예제파일/or10_5.py
Objective: 71
Route for vehicle 0:
0 Time(0,0) -> 9 Time(2,3) -> 14 Time(7,8) -> 16 Time(11,11) -> 0 Time(18,18)
Time of the route: 18min

Route for vehicle 1:
0 Time(0,0) -> 7 Time(2,4) -> 1 Time(7,11) -> 4 Time(10,13) -> 3 Time(16,16) -> 0 Time(24,24)
Time of the route: 24min

Route for vehicle 2:
0 Time(0,0) -> 12 Time(4,4) -> 13 Time(6,6) -> 15 Time(11,11) -> 11 Time(14,14) -> 0 Time(20,20)
Time of the route: 20min

Route for vehicle 3:
0 Time(0,0) -> 5 Time(3,3) -> 8 Time(5,5) -> 6 Time(7,7) -> 2 Time(10,10) -> 10 Time(14,14) -> 0 Time(20,20)
Time of the route: 20min

Total time of all routes: 82min
❖ (base) okcs@ogchangsuui-MacBookPro 01_python_예제파일 %
```

Open-tour TSP

시작 지점을 돌아올 필요가 없는 경로를 Open Tour라고 한다.

(문제) 레인보우 회사는 4가지 색깔의 페인트를 생산한다. 이 4가지를 한 장치 공정에서 생산하기 때문에 하나의 색깔을 생산하고 난 후 다음 색깔의 페인트를 생산하기 위해 청소가 필요하다. 이 청소 시간은 생산되는 색깔에 따라 다른데 그 시간은 다음과 같다.

TABLE 11.1 Interbatch Cleanup Times (in minutes) for the Paint Production Problem

<i>Paint</i>	Interbatch cleanup time (min)			
	<i>White</i>	<i>Yellow</i>	<i>Black</i>	<i>Red</i>
White	∞	10	17	15
Yellow	20	∞	19	18
Black	50	44	∞	22
Red	45	40	20	∞

이때 청소 시간의 합을 최소화하는 페인트 생산순서를 정하여라.

Open-tour TSP

- 일반적인 TSP 문제를 풀면 최적 생산 순서가 $B \rightarrow W \rightarrow Y \rightarrow R \rightarrow B$ 로 결정됨.
- 우리는 $R \rightarrow B$ 청소시간은 빼고 순서를 결정해야 함.
- 이를 위해 다음과 같이 변경 비용이 0 인 가상 페인트를 도입한 모형을 고려할 수 있음.

$$\|d_{ij}\| = \begin{pmatrix} \infty & 10 & 17 & 15 & 0 \\ 20 & \infty & 19 & 18 & 0 \\ 50 & 44 & \infty & 22 & 0 \\ 45 & 40 & 20 & \infty & 0 \\ 0 & 0 & 0 & 0 & \infty \end{pmatrix}$$

가상

TABLE 11.1 Interbatch Cleanup Times (in minutes) for the Paint Production Problem

Paint	Interbatch cleanup time (min)			
	White	Yellow	Black	Red
White	∞	10	17	15
Yellow	20	∞	19	18
Black	50	44	∞	22
Red	45	40	20	∞

- 이 모형의 최적 경로가 $W \rightarrow Y \rightarrow R \rightarrow B \rightarrow$ 가상 $\rightarrow W$ 라고 하면 이는
 가상 $\rightarrow W \rightarrow Y \rightarrow R \rightarrow B \rightarrow$ 가상 으로 해석할 수 있고 결국 최적 생산 순서는
 $W \rightarrow Y \rightarrow R \rightarrow B$ 임.

실습 1. Open-tour TSP

어떤 기계에서 다음 5개의 작업을 처리하는 순서를 정해야 한다고 하자. 각 작업의 Setup 시간은 선행 작업에 따라 다음과 같이 다르다. Setup 시간의 합을 최소화하는 작업 순서를 결정하는 수리모형을 작성하여라.

		후행 작업				
		A	B	C	D	E
선행작업	없음	4	5	8	9	4
	A	-	7	12	10	9
	B	6	-	10	14	11
	C	10	11	-	12	10
	D	7	8	15	-	7
	E	12	9	8	16	-

실습

- 예제 1, 2 (코드 제출 1, 2)
 - 예제 3 에 대한 실습지 완성 (코드 3, 4, 5 제출) → 수업시간에 배포
 - Open TSP 예제 (코드 6 제출)
-
- 위 코드 6개를 zip으로 묶어 클래스넷에 제출할 것